

Generators

Martin Schoeberl

Technical University of Denmark
Embedded Systems Engineering

September 21, 2026

Overview

- ▶ Functions with more outputs
- ▶ Solution to the min/max lab
- ▶ Type conversions
- ▶ Parameters (simple and types)
- ▶ More Scala for generators

Levels of Parameterization

- ▶ Fixed hardware
- ▶ Value parameter: `width: Int`
- ▶ Configuration object: `Config(...)`
- ▶ Type parameter: `T <: Data`
- ▶ Structural generation: `map / tabulate / reduceTree / for`
- ▶ Architecture generator: network topology, pipelines,...

Functions with Multiple Outputs

- ▶ We use functions to generate hardware
- ▶ The return value is the *output* of that *module*
- ▶ A function usually has a single return value
- ▶ Use a Scala tuple for more output *ports*

```
def compare(a: UInt, b: UInt) = {  
  val equ = a === b  
  val gt = a > b  
  (equ, gt)  
}
```

Functions with More Outputs

- ▶ Access the two output wires with the `._n` syntax.

```
val cmp = compare(inA, inB)
val equResult = cmp._1
val gtResult = cmp._2
```

- ▶ Or directly decompose the tuple

```
val (equ, gt) = compare(inA, inB)
```

- ▶ Functions can be declared as part of a Module
- ▶ Better place them into a Scala object collecting utility functions
- ▶ Functions can serve as lightweight modules

Lab Last Week

- ▶ Write a search for the minimum circuit (with `treeReduce()`)
- ▶ Add the generation of the index of the minimum value
- ▶ This was a problem formulated by Microchip

Functional Generation

- ▶ Anonymous functions, called *function literal*

```
(param) => function body
```

- ▶ A function for a minimum search

```
val min = vec.reduceTree((x, y) => Mux(x <  
    y, x, y))
```

- ▶ This was a very short exercise - let us extend this

Minimal Function with Index

- ▶ Was the example for Tjark's heap sort

```
class Two extends Bundle {  
  val v = UInt(w.W)  
  val idx = UInt(8.W)  
}  
  
val vecTwo = Wire(Vec(n, new Two()))  
for (i <- 0 until n) {  
  vecTwo(i).v := vec(i)  
  vecTwo(i).idx := i.U  
}  
  
val res = vecTwo.reduceTree((x, y) =>  
  Mux(x.v < y.v, x, y))
```

- ▶ We need an extra bundle to hold both values
- ▶ A for loop is not so functional

We Can Use Tuples and zipWithIndex

- ▶ `zipWithIndex` transforms the original sequence to a sequence of tuples with second element is the index
- ▶ Use `map` to translate from Scala `Int` to Chisel `UInt`
- ▶ `reduce` does the minimum function, actually called 2 times (we could optimize this)

```
val resFun = vec.zipWithIndex
  .map((x) => (x._1, x._2.U))
  .reduce((x, y) => (Mux(x._1 < y._1, x._1,
    y._1), Mux(x._1 < y._1, x._2, y._2)))
```

- ▶ The result is a Scala `Vector` and not a Chisel `Vec`
- ▶ No `reduceTree` available on a Scala `Vector`

Solution with a Vec

- ▶ This results in a Chisel Vec

```
val scalaVector = vec.zipWithIndex
    .map((x) => MixedVecInit(x._1,
        x._2.U(8.W)))
val resFun2 = VecInit(scalaVector)
    .reduceTree((x, y) => Mux(x(0) < y(0), x,
        y))

val minVal = resFun2(0)
val minIdx = resFun2(1)
```

- ▶ MixedVecInit is like a Bundle, but indexable
- ▶ We should add a reduceTree to the Scala sequence version (TraversableOnce)

Type Conversion

- ▶ Sometimes we would like to see a value in different types
- ▶ All types represent a collection of bits
- ▶ Example to package 4 bytes into a 32-bit UInt

```
val vec = Wire(Vec(4, UInt(8.W)))  
val word = vec.asUInt
```

- ▶ And converting it back to a Vec

```
val vec2 = word.asTypeOf(Vec(4, UInt(8.W)))
```

Type Conversion

- ▶ Convert a Bundle to a UInt

```
class MyBundle extends Bundle {  
  val a = UInt(8.W)  
  val b = UInt(16.W)  
}
```

```
val bundle = Wire(new MyBundle)  
val word2 = bundle.asUInt
```

- ▶ A UInt can be converted (back) to a bundle

```
val bundle2 = word2.asTypeOf(new MyBundle)
```

- ▶ Initialize to 0 on a conversion

```
val bundle3 = 0.U.asTypeOf(new MyBundle)
```

Simple Parameters

- ▶ Simplest way is bit width
- ▶ You have seen this, also in Verilog or VHDL

```
class ParamAdder(n: Int) extends Module {  
  val io = IO(new Bundle{  
    val a = Input(UInt(n.W))  
    val b = Input(UInt(n.W))  
    val c = Output(UInt(n.W))  
  })  
  
  io.c := io.a + io.b  
}
```

- ▶ A bit more interesting is using case classes for parameters

```
case class Config(txDepth: Int, rxDepth: Int,  
  width: Int)
```

Case Classes

- ▶ Reading the immutable fields

```
val param = Config(4, 2, 16)

println("The width is " + param.width)
```

- ▶ Adding checking code to case classes

```
case class SaveConf(txDepth: Int, rxDepth:
    Int, width: Int) {

    assert(txDepth > 0 && rxDepth > 0 && width >
        0, "parameters must be larger than 0")
}
```

Module with Type Parameters

- ▶ Assume a network-on-chip
- ▶ Moves data between processing cores
- ▶ We want to *parameterize* that data type
- ▶ Add a type parameter T to the Module constructor

```
class NocRouter[T <: Data](dt: T, n: Int)
  extends Module {
    val io = IO(new Bundle {
      val inPort = Input(Vec(n, dt))
      val address = Input(Vec(n, UInt(8.W)))
      val outPort = Output(Vec(n, dt))
    })

    // Route the payload according to the address
    // ...
```

Use that Router

- ▶ Define data type we want to route with

```
class Payload extends Bundle {  
  val data = UInt(16.W)  
  val flag = Bool()  
}
```

- ▶ Pass an instance of that bundle to the constructor of the router

```
val router = Module(new NocRouter(new  
  Payload, 2))
```

Parameterized Bundles

- ▶ We still have vectors of addresses and the payload
- ▶ We want to parametrize a Bundle

```
class Port[T <: Data](dt: T) extends Bundle {  
  val address = UInt(8.W)  
  val data = dt.cloneType  
}
```

- ▶ dt is the type parameter, which we use for cloneType
- ▶ However, it is a public field, in the way for using in a Vec

Parameterized Bundles

- ▶ As a fix (workaround) make it private

```
class Port[T <: Data](private val dt: T)
  extends Bundle {
    val address = UInt(8.W)
    val data = dt.cloneType
  }
```

- ▶ Define our router ports

```
class NocRouter2[T <: Data](dt: T, n: Int)
  extends Module {
    val io = IO(new Bundle {
      val inPort = Input(Vec(n, dt))
      val outPort = Output(Vec(n, dt))
    })
```

```
// Route the payload according to the address
// ...
```

Using Parameterized Bundles

- ▶ Instantiate that router with a Port that takes a Payload as a parameter

```
val router = Module(new NocRouter2(new  
    Port(new Payload), 2))
```

Scala Option

- ▶ Scala's `Option[T]` is a wrapper around type `T`
- ▶ Potential non-existence
- ▶ Is either `Some(x)` or `None`

```
val opt: Option[Int] = Some(123)
if (opt.isDefined)
  println(opt.get)
else
  println("None")
```

Optional Ports

- ▶ IO ports may depend on configuration
- ▶ In Scala, this is represented as an `Option`
- ▶ Return a value wrapped in `Some` or represent the missing value as a `None`
- ▶ Could be used for debugging

```
dut.io.dbgPort.get(4).expect(123.U)
```

Example: Register File

```
class RegisterFile(debug: Boolean) extends Module {  
  val io = IO(new Bundle {  
    val rs1 = Input(UInt(5.W))  
    val rs2 = Input(UInt(5.W))  
  
    ...  
    val rs2Val = Output(UInt(32.W))  
    val dbgPort = if (debug)  
      Some(Output(Vec(32, UInt(32.W)))) else None  
  })  
  val regfile =  
    RegInit(VecInit(Seq.fill(32)(0.U(32.W))))  
  io.rs1Val := regfile(io.rs1)  
  io.rs2Val := regfile(io.rs2)  
  when(io.wrEna) {  
    regfile(io.rd) := io.wrData  
  }  
  if (debug) {  
    io.dbgPort.get := regfile  
  }  
}
```

Scala tabulate

- ▶ More general than `fill`
- ▶ Produce a new collection by calling an anonymous function
- ▶ Index is the single argument
- ▶ Can use the `_` wildcard

```
Seq.fill(5)(0)
```

```
Seq.tabulate(5)(i => i * i)
```

```
Seq.tabulate(5)(_ + 1)
```

Scala's apply() Method

- ▶ Create a new instance without new

```
val p = Person("Joep Hacker")
```

- ▶ is translated during compilation to

```
val p = Person.apply("Joep Hacker")
```

- ▶ apply() is also used as access function for arrays
- ▶ No special syntax for arrays

Scala's apply() Method

- ▶ A *companion object* in Scala is an object
 - ▶ Declared in the same file as a class,
 - ▶ and has the same name as the class
- ▶ Add an apply() method to the companion object

```
class Person {  
    var name = ""  
}  
  
object Person {  
    def apply(name: String): Person = {  
        var p = new Person  
        p.name = name  
        p  
    }  
}
```

Factory Methods

- ▶ Simpler component creation and use
- ▶ Usage similar to built-in components, such as Mux

```
val myAdder = Adder(x, y)
```

- ▶ A little bit more work on the component side
- ▶ Easier for the user of the component

Factory Methods Cont.

- ▶ Define an apply method on the companion object that returns the component (output port)

```
object Adder {  
  def apply(a: UInt, b: UInt) = {  
    val adder = Module(new Adder)  
    adder.io.a := a  
    adder.io.b := b  
    adder.io.result  
  }  
}
```

- ▶ Hides module instantiation, wiring, and exposes only the result
- ▶ Important generator/library-design pattern

Use a var for a Generator

- ▶ A var can be reassigned
- ▶ Just keep the Chisel components connected
- ▶ No need to keep the references around

```
class Delay(n: Int) extends Module {  
  val io = IO(new Bundle {  
    val in  = Input(Bool())  
    val out = Output(Bool())  
  })  
  require(n >= 0)  
  var lastConn = io.in  
  for (i <- 0 until n)  
    lastConn = RegNext(lastConn)  
  io.out := lastConn  
}
```

Do you want Recursion?

- With a helper function

```
class DelayRec(n: Int) extends Module {  
  val io = IO(new Bundle {  
    val in  = Input(Bool())  
    val out = Output(Bool())  
  })  
  require(n >= 0)  
  def helper(n: Int, lastConn: Bool): Bool =  
    {  
      if (n == 0) lastConn  
      else helper(n-1, RegNext(lastConn))  
    }  
  io.out := helper(n, io.in)  
}
```

Break

- ▶ Then let us build some hardware

Parameterize Number of Ports

- ▶ Use a Vec

```
class Example(n: Int, w: Int) extends Module {  
  val io = IO(new Bundle {  
    val in  = Input(Vec(n, UInt(w.W)))  
    val out = Output(UInt(w.W))  
  })  
}
```

- ▶ The number and type of ports can be decided at elaboration time
- ▶ The interface itself can depend on generator parameters

Use Methods

- ▶ Modules and Bundles are Scala classes
- ▶ One can define methods on them, like in any Scala class
- ▶ We have seen `apply()` on the companion object
- ▶ Remember what this gave us?

Decoupled Components

- ▶ Using the read/valid handshake
- ▶ Some flexibility on sending and receiving
- ▶ Data is transferred when ready and valid are asserted
- ▶ Chisel provides a standard bundle for this: DecoupledIO

```
class DecoupledIO[T <: Data](gen: T) extends
  Bundle {
    val ready = Input(Bool())
    val valid = Output(Bool())
    val bits  = Output(gen)
  }
```

Methods on DecoupledIO

- ▶ `fire()` is true if and only if `ready` and `valid` are asserted
- ▶ `enq(data)` sets `data` and sets `valid` to true (no check on `ready`)
- ▶ `noenq()` deasserts `valid`
- ▶ `deq/nodeq` for the receiver side

```
when(io.in.fire()) {  
    // do the transfer  
}
```

- ▶ Sala convention: methods with no side effects, leaving off the empty parentheses

```
when(io.in.fire) {  
    // do the transfer  
}
```

Methods on DecoupledIO

- ▶ Distinction between a Scala method and generated hardware
- ▶ `io.in.fire` looks like a software method invocation
- ▶ But produces a Chisel `Bool` expression representing `ready && valid`.

Arbiter

- ▶ Several components have a request for a shared resource
- ▶ Examples: bus, memory port, on-chip network ports
- ▶ Needs to select which request gets granted
- ▶ Different algorithms to do this
- ▶ Simplest is priority, but unfair
- ▶ Fairness and guaranteed progress, especially in real-time systems, is an issue
- ▶ I have a fair arbiter in the Chisel book (10.6.2)

Chisel Arbiter

- ▶ Uses DecoupledIO for inputs and the output

```
val arb = Module(new Arbiter(UInt(), 2))
arb.io.in(0) <> producer0.io.out
arb.io.in(1) <> producer1.io.out
consumer.io.in <> arb.io.out
```

- ▶ Arbiter is a simple priority based arbitration
- ▶ RRArbiter chooses in round robin
 - ▶ No details on implementation
 - ▶ A single-cycle round-robin arbiter can have substantial combinational logic
- ▶ LockingRRArbiter grant reserved for several clock cycles

Crossbar

- ▶ N inputs are connected to M outputs
- ▶ Needs M arbitration circuits, each of N inputs
- ▶ Need an address for the destination
- ▶ Scales only to a moderate N and M
- ▶ Add pipelining for larger N and M
- ▶ Switch to a Network-on-Chip

FIFO Queue

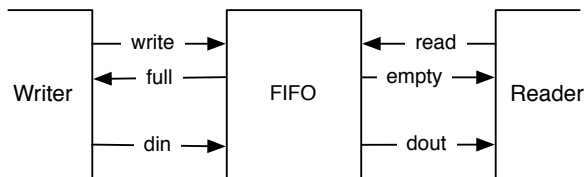


Figure: A writer, a FIFO buffer, and a reader.

- ▶ FIFO style queues between a sender and a receiver
- ▶ The sender/writer enqueues/writes into the Queue
- ▶ The receiver/reader dequeues/reads from the Queue
- ▶ Can *even out* bursty traffic

Queues

- ▶ Used when there is bursty traffic
 - ▶ E.g., on a serial port
 - ▶ Write faster from CPU than serial can send
 - ▶ Receive data when CPU is busy with other stuff
- ▶ But can fill up when the sender is continuously *faster*
- ▶ Showing implementation variations in the Chisel book
 - ▶ Bubble FIFO (was proposed in lab 5)
 - ▶ double buffer, memory-based, ...

Bubble FIFO Example

- ▶ FIFO interface

```
class FifoIO[T <: Data](private val gen: T)
  extends Bundle {
    val enq = Flipped(new DecoupledIO(gen))
    val deq = new DecoupledIO(gen)
  }
```

- ▶ Abstract base class (with some common code)

```
abstract class Fifo[T <: Data](gen: T, val
  depth: Int) extends Module {
  val io = IO(new FifoIO(gen))

  require(depth > 0, "Number of buffer
    elements needs to be larger than 0")
}
```

- ▶ FIFO code

Bubble FIFO

- ▶ Simple, easy to understand
- ▶ Uses minimal resources
- ▶ However, each buffer stage has to toggle between empty and full
- ▶ Maximum bandwidth is one word every two clock cycles
- ▶ When full, needs N cycles for a restart
 - ▶ The free element *bubbles* towards the input
- ▶ Better solutions
 - ▶ Double buffer
 - ▶ Memory with read and write pointers

Use Inheritance

- ▶ Defined an abstract base class for the FIFO and extend it
 - ▶ Also use traits to inherit from several sources
- ▶ Learn from software development to share code
- ▶ We select the implementation by the type

```
class DoubleBufferFifo[T <: Data](gen: T,  
    depth: Int) extends Fifo(gen: T, depth:  
    Int) {  
    ...  
class RegFifo[T <: Data](gen: T, depth: Int)  
    extends Fifo(gen: T, depth: Int) {  
    ...  
class MemFifo[T <: Data](gen: T, depth: Int)  
    extends Fifo(gen: T, depth: Int) {  
    ...
```

- ▶ Explore the different implementation details in Section 11.3 of the Chisel book

Chisel Queue

- ▶ Chisel has a Queue
- ▶ Interfaces are DecoupledIOs
- ▶ Specify type and number of entries Queue(UInt(4.W), 8)
- ▶ Optional arguments
 - ▶ pipe: if full, allow concurrent enqueue and dequeue
 - ▶ flow: if empty, enqueued value available immediately for dequeue
- ▶ What is the implementation? Resources? Throughput?

```
val q = Module(new Queue(UInt(), 16))  
q.io.enq <> producer.io.out  
consumer.io.in <> q.io.deq
```

Connect a Network-on-Chip

- ▶ $n \times n$ router in bi-thorus configuration
- ▶ Each router has 4 + 1 ports: North, South, West, East, and local
- ▶ Use a helper function `connect(...)`

```
def connect(r1: Int, p1: Int, r2: Int, p2: Int):  
    Unit = {  
        net(r1).io.ports(p1).in :=  
            net(r2).io.ports(p2).out  
        net(r2).io.ports(p2).in :=  
            net(r1).io.ports(p1).out  
    }  
  
for (i <- 0 until n) {  
    for (j <- 0 until n) {  
        val r = i * n + j  
        connect(r, EAST, i * n + (j + 1) % n, WEST)  
        connect(r, SOUTH, (i + 1) % n * n + j, NORTH)  
    }  
}
```

Now we are Generating

- ▶ The NoC example is a real generator
- ▶ This is what we have been working toward
- ▶ We aren't just parameterizing widths anymore
- ▶ Scala is generating the architecture
- ▶ 11 lines of Scala generate an entire network structure

Generator Summary

- ▶ Scala runs at elaboration time
- ▶ Chisel objects describe the hardware being constructed
- ▶ Generators turn parameters, types, and algorithms into hardware structure

Why Open Source?

- ▶ Can improve your visibility
- ▶ Add your GitHub repos to your CV
- ▶ When hiring, I look up GitHub contributions
- ▶ You can get help from the community
- ▶ I got free translations of my Chisel book
- ▶ Why not?
 - ▶ You already use open-source tools and libraries
 - ▶ So return some to the community
 - ▶ Even in a company, if there is no patent
 - ▶ Oticon contributes to Zephyr

Open-Source Licensing

- ▶ You own the copyright when creating code
 - ▶ Hardware is different in DK, so it is blurry
 - ▶ Maybe your employer owns the copyright (not at DTU)
- ▶ A license grants others to use and change your code
 - ▶ Different licenses have different permissions and restrictions
- ▶ When releasing, add a license to your repo
 - ▶ Have it clearly visible (e.g., as LICENSE in the project root)
 - ▶ GitHub will detect it

License Types

- ▶ BSD and MIT
 - ▶ Commonly used, good for academic
 - ▶ Basically, use my stuff, but don't sue me
 - ▶ Can also be used in a commercial product (e.g., network stack)
- ▶ GPL
 - ▶ Commonly used in software
 - ▶ Distributing derived software can require making corresponding source available under the GPL
 - ▶ Some companies have restrictions on using GPL software
- ▶ Apache
 - ▶ Similar to BSD and MIT
 - ▶ Includes explicit patent protection

Documentation

- ▶ Summarize the purpose of your project
- ▶ Instruct how to use it
- ▶ List dependencies
- ▶ Good documentation:
 - ▶ May attract users
 - ▶ May even attract contributors
 - ▶ Forces you to rethink your project
- ▶ Start early with the documentation

README.md

- ▶ Well-suited for small to medium projects
- ▶ Immediately visible on GitHub
- ▶ Use Markdown for simple formatting
- ▶ Include figures, best in SVG
- ▶ I will consider the README as part of the grade
 - ▶ Don't wait till the final week!

On the Project

- ▶ Pick a topic and fill in the Google Doc
- ▶ Have some inspiration from <https://scottbeamer.net/cse228a-sp25/project/index.html>
- ▶ Next week is your initial presentation
- ▶ No lab

The Exam and Grading

- ▶ Grade is based on:
 - ▶ The project (source code)
 - ▶ The documentation
 - ▶ The presentations
- ▶ Final week: your project presentation (= exam)
- ▶ No report to be written
- ▶ But I expect a very detailed README.md
 - ▶ Explain what it does and how to use it
 - ▶ State your usage of AI/LLM

Summary

- ▶ We use Scala to write hardware generators
- ▶ Get started with your project
- ▶ Collect ideas for more productive generator code
- ▶ Did you find some Scala tricks that I did not present?
 - ▶ Add to our shared Google Docs
 - ▶ Also, what you are missing
- ▶ Use ready/valid interfaces (DecoupledIO)
- ▶ Use queues, arbitration
- ▶ Go open source!
- ▶ Next week: guest lecture by Ioannis on LLM usage at Microhip